

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

ML-POWERED INTRUSION DETECTOR

Project Report

Submitted by: Shreya Yadav (24BCE0171)

Anwesha Laha (24BCI0024)

Khushi Bhalla (24BCI0026)

Course Name: Computer Networks Lab

Faculty Name: Sreethar S

Table of Contents

Section	Contents	Page
-	Abstract	3
1	Introduction	4
2	Problem Statement	4
3	Objectives	4
4	System Overview and Architecture	4
5	Part I - Supervised Intrusion Detection	5
6	Part II - Zero-Day Anomaly Detection	5
7	Part III - SOC Dashboard and Incident Reporting	5
8	Incident Correlation and Response Workflow	13
9	Implementation Technologies	13
10	Testing and Evaluation	14
11	Results and Discussion	14
12	Limitations and Future Enhancements	14
13	Conclusion	14
	References	15

Abstract

Modern computer networks face a growing range of security threats, including denial-of-service (DoS) attacks, port scanning, credential brute-force attempts, and previously unseen patterns of malicious or anomalous traffic. Traditional signature-based intrusion detection can identify known attack patterns, but it may struggle when an attack changes its behaviour or does not match an existing signature. This project presents an ML-powered intrusion detection system designed to combine supervised classification, unsupervised anomaly detection, and a Security Operations Center (SOC) dashboard into one operational workflow. Its purpose is to support the identification of known threats, highlight unusual traffic for further investigation, and present alerts in a form that is useful to security analysts.

The supervised detection component uses Random Forest and HistGradientBoosting classifiers to categorize network flows into classes such as Benign, DoS, PortScan, and BruteForce. Model development uses the CIC-IDS2017 dataset, with cross-dataset testing against CSE-CIC-IDS2018 to examine how well the learned patterns transfer to a different data environment. To complement the supervised models, the anomaly-detection component uses a Deep Autoencoder and Isolation Forest. The Autoencoder learns a representation of normal traffic and uses reconstruction error to flag flows that differ from that baseline, while Isolation Forest identifies observations that can be isolated from the majority of the data. This second detection path is intended to surface suspicious behaviour even when a specific attack label is not available, although such alerts still require investigation and careful threshold calibration.

Predictions and detections are converted into structured alerts and delivered to a FastAPI backend, where requests are validated and records are stored in SQLite. A React-based dashboard provides a central view of security telemetry, alert severity, source-IP activity, incident correlation, and response recommendations. Supporting features include manual alert ingestion, raw syslog parsing, temporal clustering of related alerts, and generation of PDF forensic reports that summarize evidence and suggested actions. These features connect model output with analyst-facing workflows rather than treating classification scores as the final response to an incident.

Reported experiments show approximately 99.94% held-out accuracy for both supervised models on CIC-IDS2017. However, cross-dataset evaluation on CSE-CIC-IDS2018 reveals generalization challenges, emphasizing that high accuracy on one test split does not guarantee equivalent performance in a different network environment. The Autoencoder achieved a ROC-AUC of 0.936 and detected 52.9% of attack flows at an approximately 1% normal-traffic alert rate under the reported evaluation setting. Overall, the project demonstrates the practical value of combining supervised and unsupervised methods with a centralized SOC interface, while also underscoring the need for cross-environment validation, false-positive monitoring, threshold tuning, and human review before taking containment actions.

1. Introduction

Network intrusion detection monitors traffic and identifies activity that may compromise confidentiality, integrity, or availability. Signature-based approaches can be effective for known patterns, but they may miss novel or modified attacks. Machine learning can learn statistical patterns from traffic features and help detect both known and unfamiliar threats. However, model predictions are most useful when they are converted into actionable alerts and interpreted in operational context. This project therefore combines detection models with a centralized SOC-style interface for reviewing telemetry, grouping related alerts, investigating source addresses, and producing incident reports.

2. Problem Statement

The project addresses three connected challenges: (1) classifying network flows into known intrusion categories with high accuracy; (2) identifying suspicious flows that differ from normal traffic without requiring attack labels during training; and (3) consolidating alerts from multiple detectors into a usable incident-management workflow. A further challenge is that high test accuracy on one dataset does not guarantee reliable performance on a different dataset or live network.

3. Objectives

- Build supervised models to classify benign and malicious network flows.
- Build an anomaly detector trained primarily on benign traffic.
- Evaluate models on held-out data and perform cross-dataset testing.
- Deliver structured JSON alerts to a central service.
- Provide a dashboard for telemetry, severity, incident correlation, and source/target analysis.
- Parse unstructured security logs and validate manually ingested alerts.
- Generate concise incident summaries, evidence, recommended actions, and PDF reports.
- Test core API, correlation, security-check, and reporting functionality.

4. System Overview and Architecture

The solution is organized as a three-part pipeline. First, flow records are preprocessed and scored by supervised and anomaly-detection models. Second, the models emit structured alerts containing fields such as source IP, destination IP, destination port, protocol, classification, confidence, and severity. Third, the backend validates and stores alerts, deduplicates repeated events where appropriate, and correlates alerts over a configurable time window. The React dashboard visualizes aggregate metrics and supports deeper investigation. A simplified flow is:

Network flow data / security logs → preprocessing and feature engineering → supervised classifier + anomaly detector → structured JSON alerts → FastAPI validation and SQLite storage → temporal correlation → React SOC dashboard and incident reports.

5. Part I — Supervised Intrusion Detection

Dataset and labels: CIC-IDS2017 is used for model development, with cross-dataset testing against CSE-CIC-IDS2018. The classification task covers Benign, DoS, PortScan, and BruteForce traffic.

Models: Random Forest and HistGradientBoosting are used as supervised classifiers. Random Forest combines predictions from multiple decision trees, while HistGradientBoosting builds boosted trees using histogram-based feature handling.

Pipeline: The workflow includes data loading, cleaning, preprocessing, feature selection, model training, held-out evaluation, and cross-dataset testing. Predictions are converted into structured JSON alerts and forwarded to the SOC dashboard.

Reported result: Both models achieved approximately 99.94% accuracy on the held-out CIC-IDS2017 test set. This is a strong result on the in-dataset test split, but accuracy alone does not describe performance for every class. Confusion matrices, per-class precision/recall, F1-score, and false-positive rates should also be reviewed. Cross-dataset tests on CSE-CIC-IDS2018 showed poor generalization for some attack patterns, demonstrating dataset shift and the importance of testing on traffic collected under different conditions.

6. Part II — Zero-Day Anomaly Detection

The anomaly-detection module aims to identify unusual traffic without requiring predefined labels for each attack. It uses CIC-IDS2017 and trains primarily on benign flows after preprocessing and feature scaling.

Deep Autoencoder: An autoencoder learns to reconstruct normal traffic features. An unusually large reconstruction error can indicate a flow that differs from the learned normal profile.

Isolation Forest: This method isolates observations through random partitioning; unusual observations tend to be isolated in fewer splits and can receive higher anomaly scores.

Pipeline: Preprocess records, scale features, train on normal-only traffic, compute anomaly scores, select a threshold, and emit alerts labelled `Anomaly`. Alerts may include confidence-like scores and the features contributing most to the anomaly.

Reported result: The Autoencoder achieved ROC-AUC 0.936 and detected 52.9% of attack flows at approximately a 1% normal-traffic alert rate. This represents a useful detection/false-alarm trade-off, but it also means that a substantial fraction of attacks were not detected at that operating point. Detection rates can vary by attack type, so thresholds should be tuned to the operational cost of false positives and missed attacks.

7. Part III — SOC Dashboard and Incident Reporting

Frontend: React.js, Vite, and Recharts provide the user interface and visualizations. Backend: FastAPI provides API endpoints, while SQLite stores telemetry and related incident data. ReportLab is used for PDF generation.

The screenshots show a dark-themed SOC interface with an executive overview, security-check workflows, live alert ingestion, raw syslog parsing, correlated incidents, telemetry feeds, and forensic reporting. The overview includes KPIs for ingested alerts, correlated incidents, critical/high incidents, live ML detections, synthetic threat seeds, and attacking source IPs. Charts summarize threat-vector distribution, hostile source IPs, targeted destination hosts, and incident severity.

Key capabilities include:

- Real-time structured alert ingestion with schema validation and deduplication.
- Configurable temporal correlation windows (shown at 5, 15, 30, and 60 minutes).
- Incident tables showing source IP, flow/alert volume, severity, risk score, attack vectors, and target scope.
- Threat-actor/source-IP investigation with optional target filtering and IP-range classification.
- Security-log parsing for SSH brute-force, firewall-drop, and web-server/WAF examples.

- Manual alert ingestion with protocol, port, attack-vector, severity, and ML-confidence fields.
- Telemetry filtering by source, classification, and detection origin.
- Incident summaries and forensic evidence, including observed endpoints, ports, classification breakdowns, and recommended containment actions.
- PDF report generation for offline review and record keeping.

Dashboard Screenshots

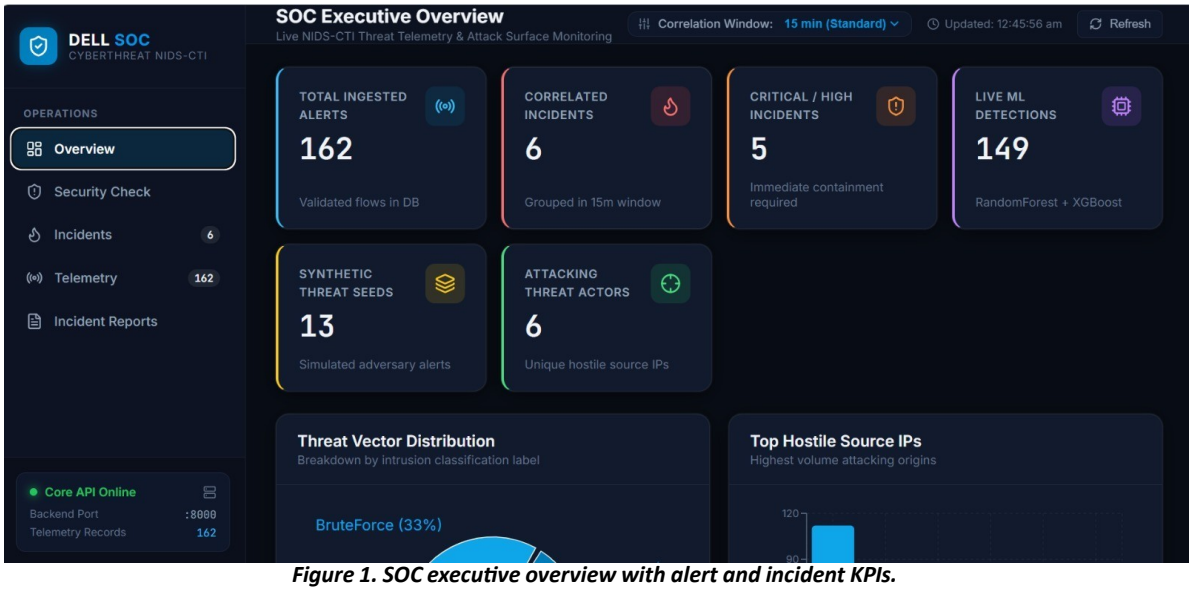


Figure 1. SOC executive overview with alert and incident KPIs.

This overview acts as the analyst's landing page. KPI cards summarize the current alert and incident workload, while the charts help an operator quickly assess whether activity is rising, which severity levels dominate, and where attention is needed first. These values represent the telemetry displayed in the supplied dashboard view; they should be interpreted as a snapshot rather than proof of live production monitoring.



Figure 10. Dashboard threat distribution and top source-IP visualizations.

These charts expand the overview by showing how detected events are distributed across traffic classes and which source addresses contribute the most activity. The distribution chart helps analysts compare benign and suspicious telemetry at a

glance, while the top-source chart highlights addresses that may warrant prioritization. These are triage aids: a high-volume source should be investigated in context rather than treated as proof of malicious intent.

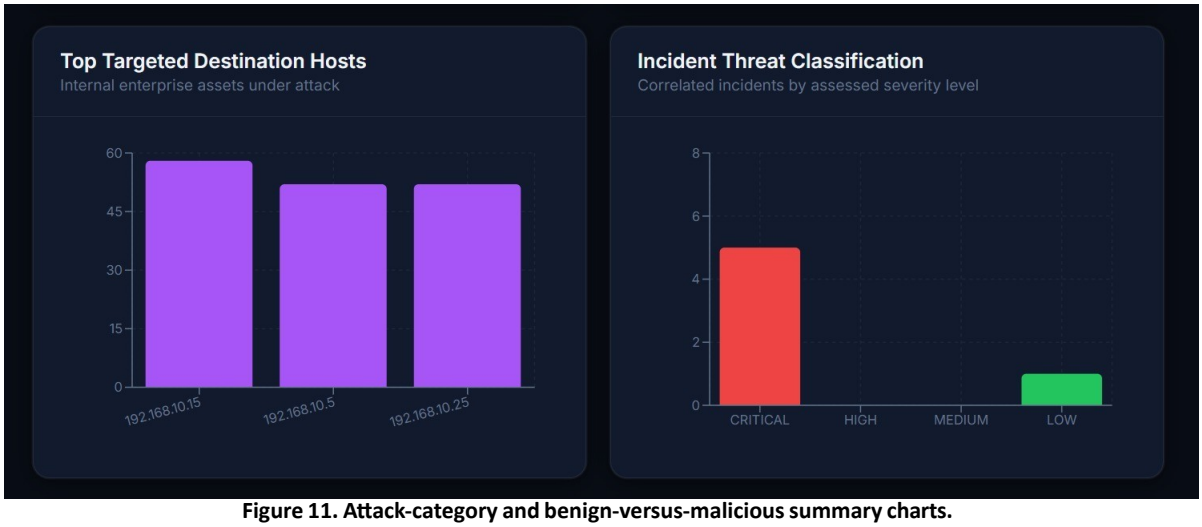


Figure 11. Attack-category and benign-versus-malicious summary charts.

The category chart compares the volume of the most prominent attack classifications, while the adjacent comparison visual separates malicious detections from benign activity. Together, they help an operator identify dominant patterns and decide whether to inspect a specific class, review model thresholds, or examine a sudden change in alert volume.



Figure 2. Correlated security incidents and severity distribution.

The incident view groups related alerts into cases and presents severity distribution so analysts can prioritize high-risk clusters. Correlation reduces the need to investigate every alert independently; the incident list provides a starting point for examining shared sources, affected targets, alert volume, and the overall risk score.

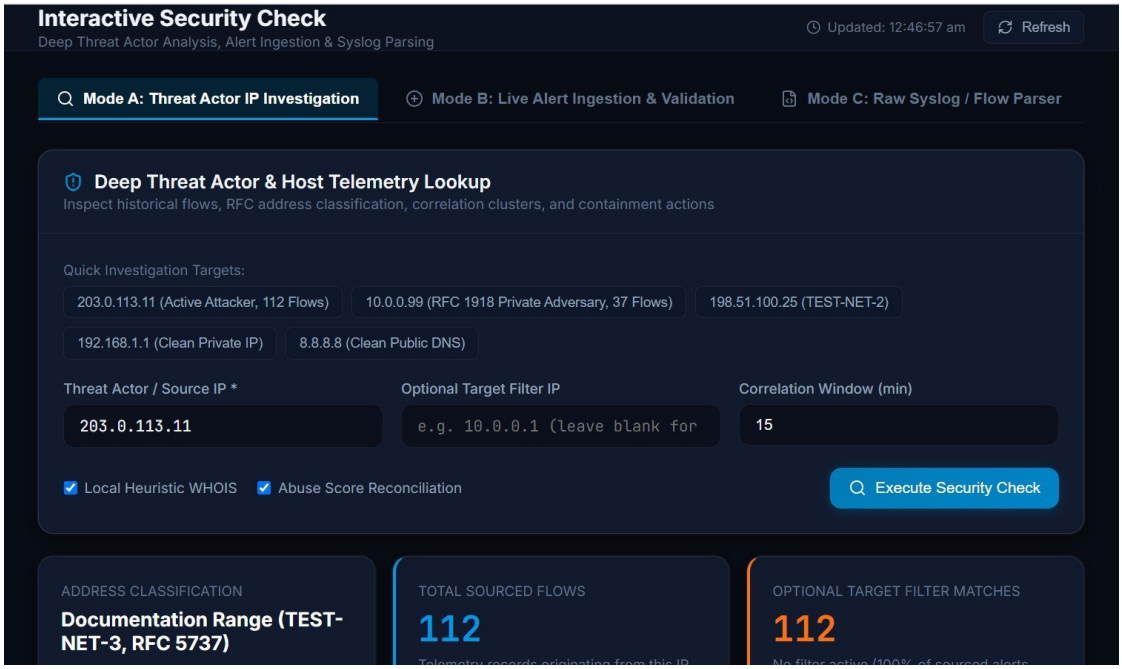


Figure 3. Interactive source-IP security investigation.

The source-IP investigation view supports pivoting from an alert to the address associated with the activity. Analysts can compare source behavior, review related flows or incidents, and apply target filters where available. This helps determine whether a source is producing repeated scans, brute-force attempts, or other suspicious patterns before deciding on containment.

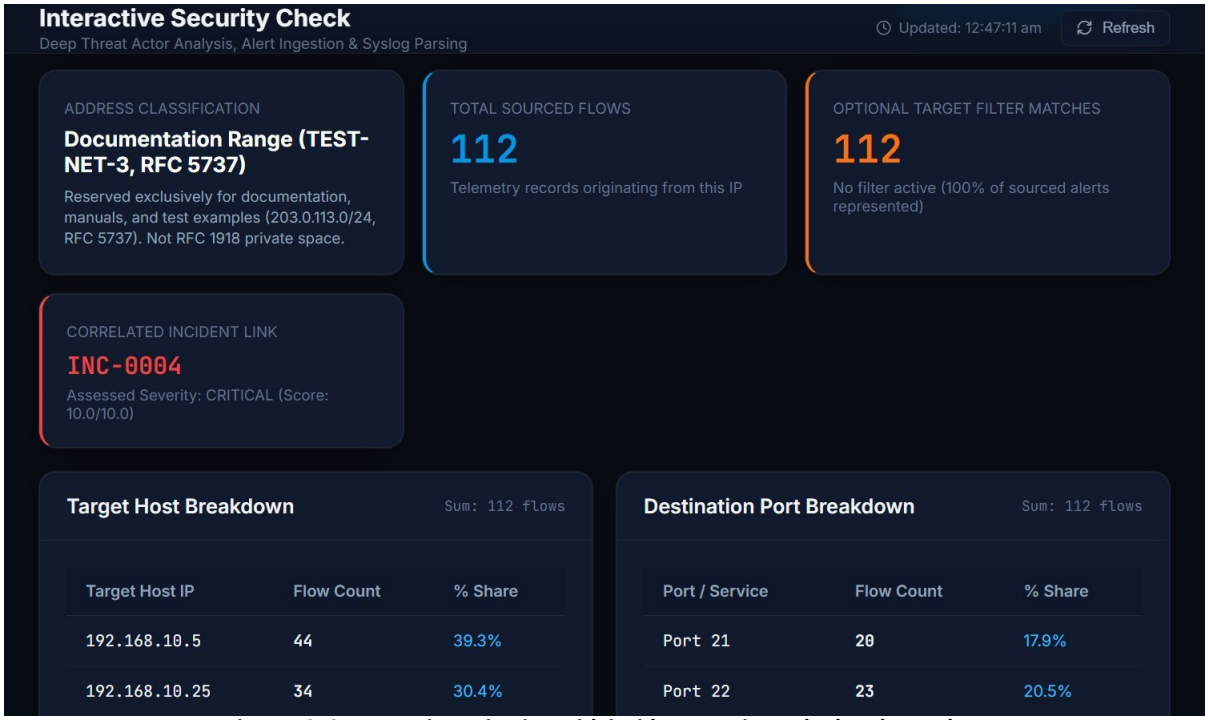


Figure 12. Source-IP investigation with incident metrics and related records.

This view provides a more detailed investigation workspace for a selected source address. Summary cards expose the number of associated alerts and risk indicators, while the record tables allow the analyst to move from a high-level source assessment to individual telemetry. This supports evidence-led triage and helps distinguish repeated activity from isolated events.



Figure 13. Source-IP investigation tables for targeted flows and detection providers.

The tables break the investigation into targeted flow records and detection-provider results. Flow-level information helps analysts verify which communications contributed to the source's risk profile; provider-level results show how different detection components characterized the activity. Comparing these views can reveal agreement, disagreement, or gaps that need manual review.



Figure 14. Detailed source-IP alert list with severity and event context.

This table presents individual alerts associated with source addresses, including severity labels and contextual event fields. Analysts can use it to compare multiple events, inspect recurring indicators, and prioritize the records most likely to affect an incident assessment. Severity is a prioritization signal and should be corroborated with the underlying evidence.

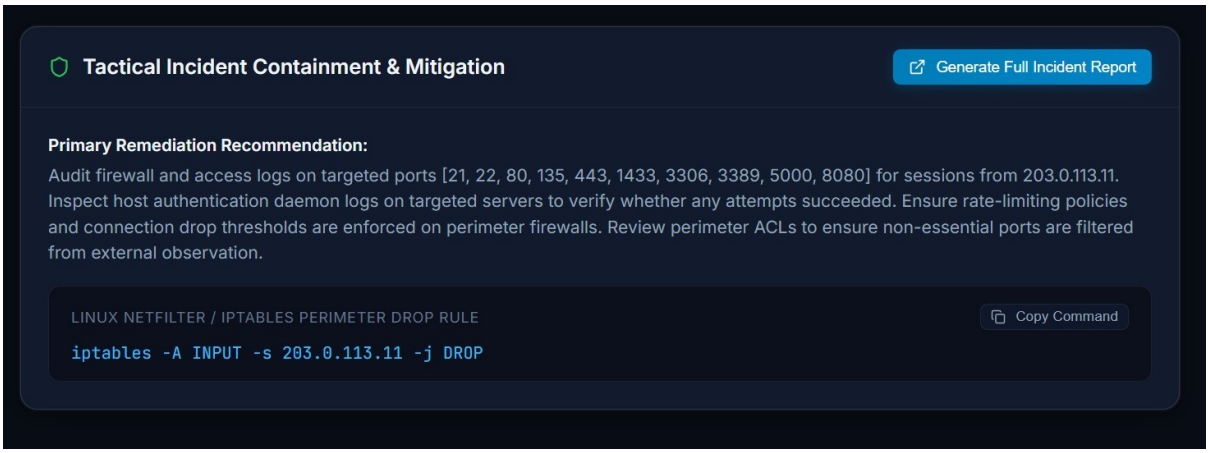


Figure 4. Tactical incident containment and mitigation recommendations.

This panel translates observed evidence into practical response steps, such as reviewing authentication and firewall logs, applying rate limits, and restricting unnecessary exposed ports. Recommendations are decision support, not automatic remediation: an analyst should validate the evidence and follow the organization's change-control process before applying network blocks or host changes.

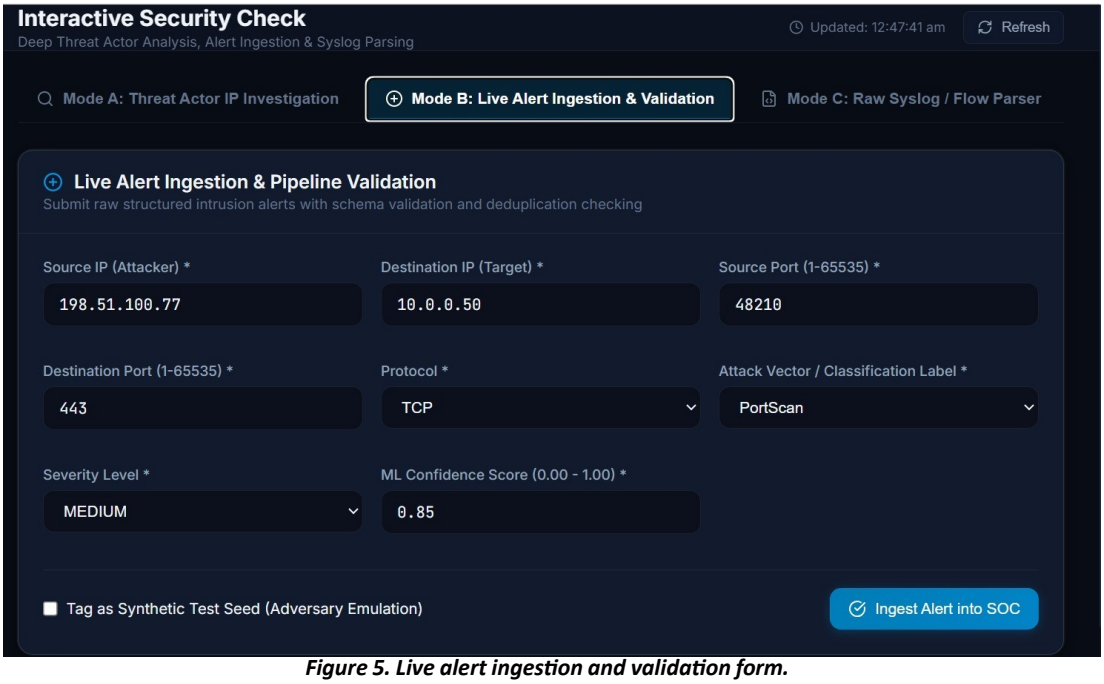


Figure 5. Live alert ingestion and validation form.

The ingestion form allows a structured alert to be submitted with fields such as source and destination, protocol, port, attack vector, severity, and model confidence. Server-side schema validation helps reject malformed records, while deduplication can reduce repeated entries. Confidence is a model signal and should not be treated as a calibrated probability unless calibration has been evaluated.

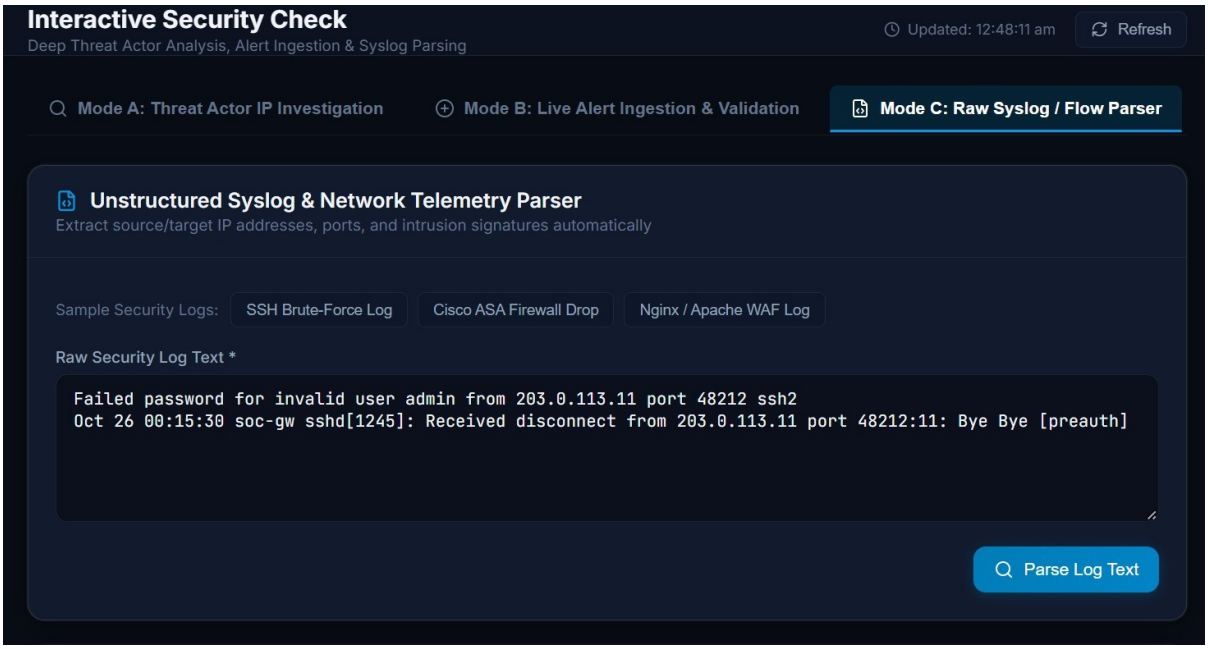


Figure 6. Raw syslog and network telemetry parser.

The parser accepts unstructured log text and extracts useful security fields into a normalized event. Example inputs can include SSH authentication failures, firewall drops, and web-server or WAF messages. Normalization makes different log sources easier to search and correlate, although parser output should be checked when logs are incomplete or use an unfamiliar format.

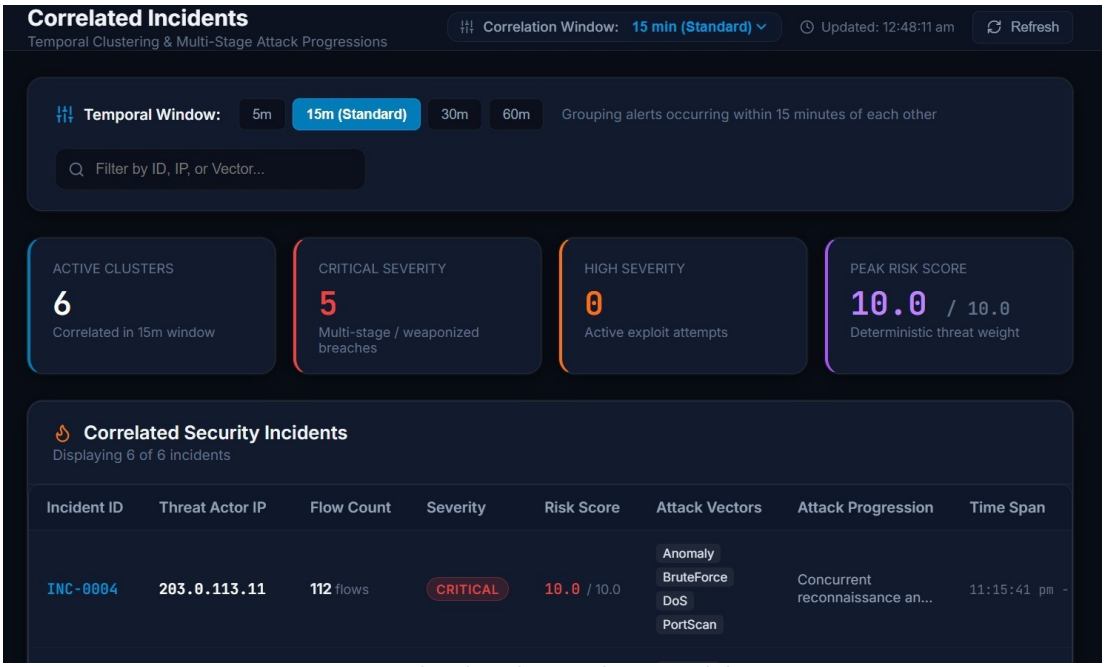


Figure 7. Correlated incidents with temporal clustering.

The correlation controls illustrate configurable time windows (for example, 5, 15, 30, or 60 minutes). Alerts occurring close together and sharing indicators such as a source address or target can be grouped into one incident. Short windows may miss slow campaigns; long windows can combine unrelated events, so the setting should match the network's traffic patterns and response needs.

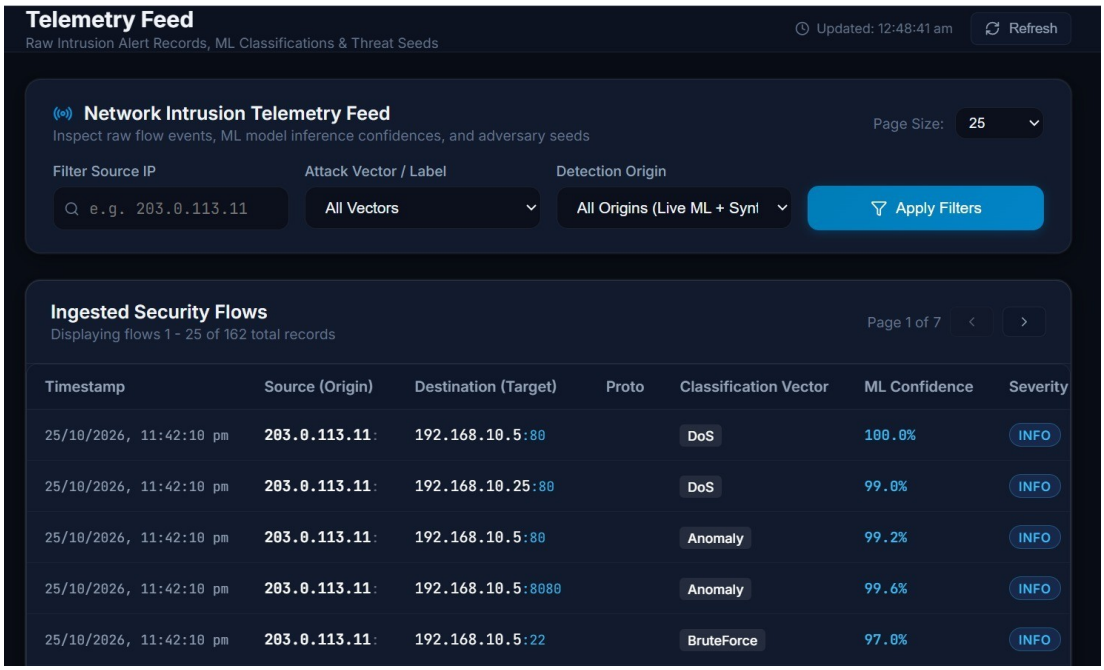


Figure 8. Telemetry feed with model classifications and confidence.

The telemetry feed provides a row-level view of events and their classifications, including whether an event is benign, a known attack category, or an anomaly. Filters by source, classification, or detection origin help narrow investigations. Model labels and confidence scores should be interpreted alongside raw features and contextual evidence, particularly when traffic differs from the training dataset.

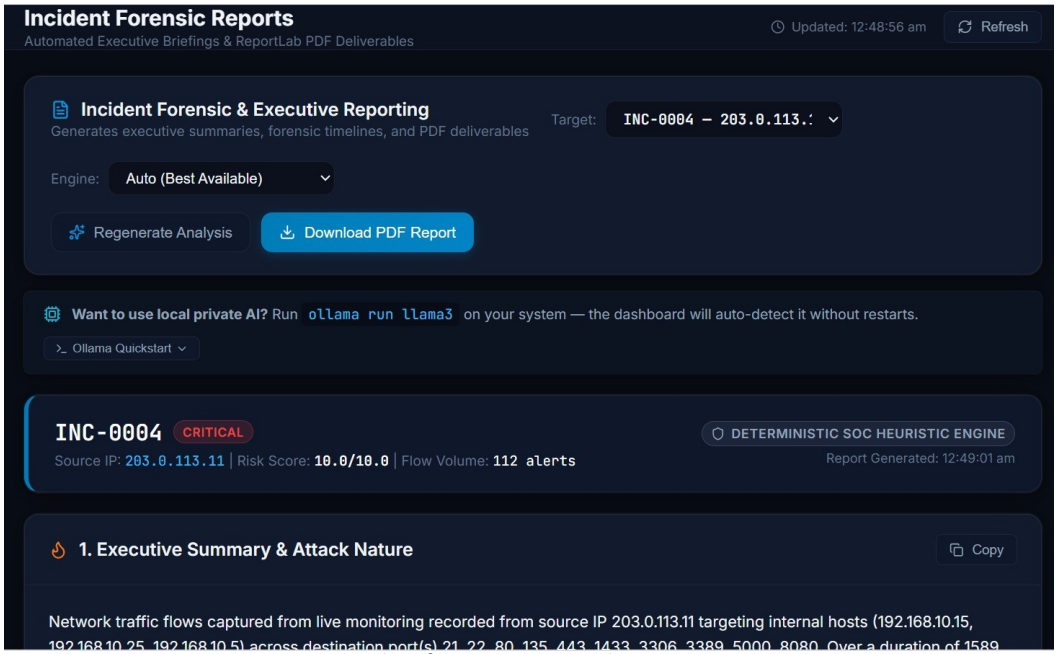


Figure 9. Incident forensic report and executive summary.

The forensic report consolidates the incident timeline, endpoints, ports, classification breakdown, supporting evidence, and recommended actions into a reviewable summary. Exporting a PDF supports hand-off, audit trails, and offline review. The report is only as reliable as the alerts and logs supplied to the system, so analysts should preserve source records and document any manual interpretation.

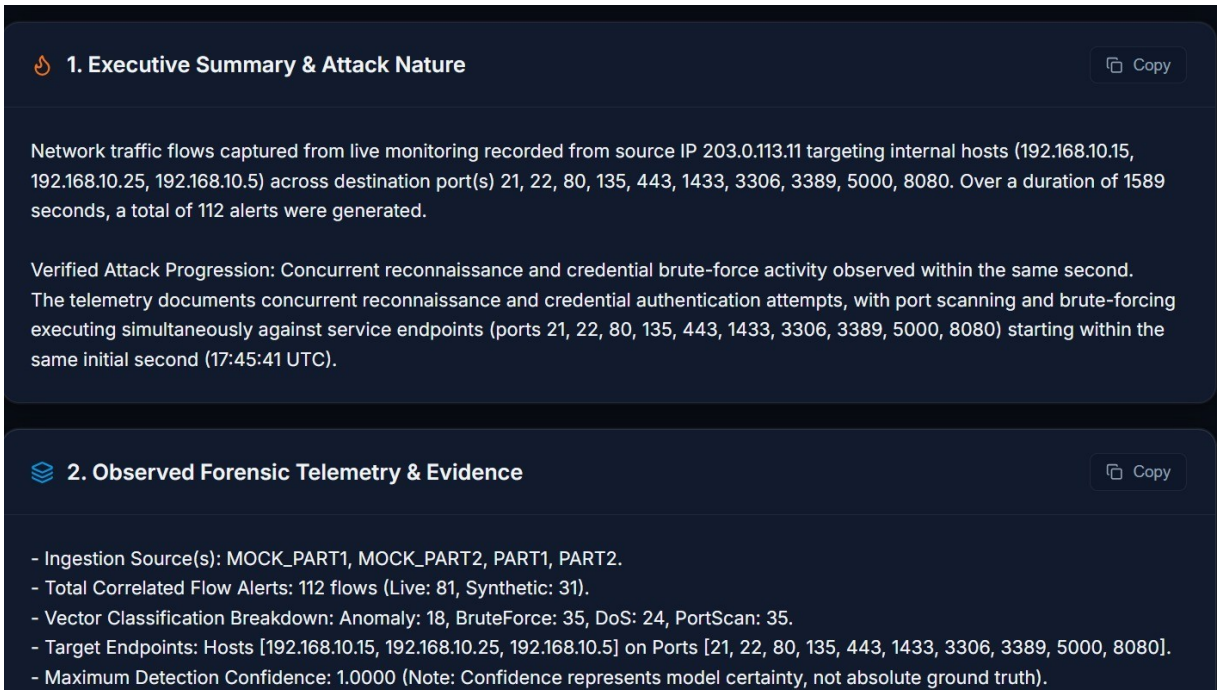


Figure 15. Executive summary and analyst notes in the forensic report.

This final report view combines an incident summary with analyst-facing notes and supporting evidence. The executive summary communicates the main finding in a compact form, while the notes section records context and recommended follow-up for later review. This makes the investigation easier to hand off and supports consistent documentation, while leaving final containment decisions to an authorized analyst.

8. Incident Correlation and Response Workflow

The correlation engine groups related alerts that occur within a selected time window. Correlation uses temporal proximity and shared indicators such as source IP, targets, and attack vectors to help analysts see a multi-stage campaign rather than isolated events. The screenshots illustrate six correlated incidents, including five marked Critical and one Low, with a peak risk score of 10.0/10.0. One displayed incident contains 112 alerts/flows associated with source 203.0.113.11 and several internal targets. These dashboard values are illustrative system telemetry shown in the supplied screenshots, not independent evidence of a real external attack.

The incident-response panel recommends reviewing firewall and access logs, checking host authentication logs, enforcing rate limits, and filtering non-essential ports. It also presents an example Linux iptables drop rule. Any containment command must be reviewed and authorized by the responsible administrator before execution; incorrect rules can interrupt legitimate traffic. In particular, the screenshot IP addresses include documentation/test ranges, so they should be treated as demonstration data unless independently verified.

9. Implementation Technologies

- Python: data preparation, model training, inference, and supporting backend logic.
- scikit-learn: Random Forest, HistGradientBoosting, Isolation Forest, preprocessing, and evaluation utilities.
- Deep learning framework: implementation of the Deep Autoencoder (the exact framework is not specified in the supplied project details).
- React.js and Vite: dashboard frontend and development/build tooling.
- Recharts: interactive data visualizations.
- FastAPI: API endpoints for alert ingestion, validation, investigation, and reporting workflows.

- SQLite: lightweight persistence for telemetry and incident records.
- ReportLab: PDF report generation.
- JSON: common interchange format between the detection pipeline and the SOC backend.

10. Testing and Evaluation

The project includes 25 unit, API, and integration tests covering alert ingestion, correlation, security checks, and report generation. This testing scope helps verify that the modules work together and that key workflows remain functional. A complete evaluation should additionally document test coverage, boundary cases, malformed input handling, duplicate-alert behavior, database failure recovery, API authentication/authorization, and load testing under realistic event rates.

Model evaluation should include confusion matrices, precision, recall, F1-score, ROC-AUC, false-positive rate, and per-class results. Cross-dataset testing is particularly important because the observed drop in generalization shows that a single random held-out split may overstate real-world readiness.

11. Results and Discussion

The project demonstrates a practical architecture that combines two complementary detection strategies with an operational dashboard. Supervised classification reports approximately 99.94% accuracy for both Random Forest and HistGradientBoosting on CIC-IDS2017 held-out data. The anomaly detector reports ROC-AUC 0.936 and 52.9% attack-flow detection at around a 1% normal-traffic alert rate. These numbers should be interpreted alongside their evaluation conditions and not as a guarantee of production performance.

The main technical finding is the difference between in-dataset performance and cross-dataset robustness. A model can learn dataset-specific patterns that do not persist across collection environments, feature distributions, or attack implementations. A layered detector can improve coverage, but it introduces the need to calibrate thresholds, normalize alert formats, manage duplicates, and prioritize alerts so analysts are not overwhelmed.

12. Limitations and Future Enhancements

Current limitations include dataset shift between CIC-IDS2017 and CSE-CIC-IDS2018, incomplete attack detection at the selected anomaly threshold, and the possibility of false positives or false negatives. Dashboard KPIs and simulated threat seeds are useful for demonstrating workflows but should not be mistaken for a live production SOC without validation against real telemetry. IP reputation or WHOIS-style classifications are contextual signals and should not independently determine whether an address is malicious.

Future work could include: (1) broader cross-dataset and time-based validation; (2) per-class threshold calibration and drift monitoring; (3) explainability for model decisions; (4) authenticated and encrypted API communication; (5) role-based access control and audit trails; (6) scalable storage and message queues for high-volume telemetry; (7) alert feedback loops for analyst labels; (8) safe, approval-based response automation; and (9) operational metrics such as time-to-detect, time-to-triage, false alerts per hour, and resource consumption.

13. Conclusion

The ML-Powered Intrusion Detector integrates supervised classification, unsupervised anomaly detection, and a centralized SOC dashboard into a single workflow. The supervised models provide strong held-out accuracy on CIC-IDS2017, while the Autoencoder adds the ability to flag suspicious patterns outside predefined attack labels. The

dashboard translates model output into correlated incidents, visual summaries, investigation tools, and PDF reports. Cross-dataset generalization remains a key challenge, and the anomaly detector's operating point leaves room for improved attack coverage. The project provides a strong foundation for further experimentation in computer networks and applied cybersecurity, provided that evaluation limitations and operational safeguards are clearly documented.

References

1. Canadian Institute for Cybersecurity. CIC-IDS2017 dataset and documentation. University of New Brunswick.

2. Canadian Institute for Cybersecurity. CSE-CIC-IDS2018 dataset and documentation. University of New Brunswick.

3. Breiman, L. "Random Forests." Machine Learning, 45, 5-32 (2001).

4. scikit-learn documentation. RandomForestClassifier, HistGradientBoostingClassifier, IsolationForest, preprocessing, and model evaluation.

5. Hinton, G. E., and Salakhutdinov, R. R. "Reducing the Dimensionality of Data with Neural Networks." Science, 313(5786), 504-507 (2006).

6. FastAPI documentation; React documentation; Vite documentation; Recharts documentation; ReportLab documentation.